INSTITUTO FEDERAL
São Paulo
Câmpus Votuporanga

ESTRUTURA DE DADOS 2

Lista de Exercícios para Revisão - P1

GUSTAVO LUIZ GORDONI

VOTUPORANGA
MARÇO / 2026

Sumário

Respostas das questões de 1 a 53 (assinalar)....4
Método Bolha....5
Fundamento....5
Complexidade....5
Etapa = 1....5
i = 0....5
i = 1....5
i = 2....6
i = 3....6
i = 4....6
i = 5....7
Etapa 2....7
i = 0....7
i = 1....7
i = 2....7
i = 3....8
i = 4....8
Etapa 3....8
i = 0....8
i = 1....9
i = 2....9
i = 3....9
Etapa 4....9
i = 0....9
i = 1....10
i = 2....10
Etapa 5....10
i = 0....10
i = 1....10
Etapa 6....10
Método Selection....12
Fundamento....12
Complexidade....12
Etapa 1 (i = 0)....12
j = 1....12
j = 2....12
j = 3....13
j = 4....13
j = 5....13
Etapa 2 (i = 1)....13
j = 2....13
j = 3....14
j = 4....14
j = 5....14
Etapa 3 (i = 2)....14
j = 3....14
j = 4....15
j = 5....15
Etapa 4 (i = 3)....15
j = 4....15

j = 5.....15
Etapa 5 (i = 4).....16
j = 5.....16
Final da etapa:.....16
Etapa 6.....16
Método Insertion.....17
Fundamento.....17
Complexidade.....17
Etapa 1 (i = 1).....17
j = 0.....17
Etapa 2 (i = 2).....17
j = 1.....18
j = 0.....18
Etapa 3 (i = 3).....18
j = 2.....18
j = 1.....19
j = 0.....19
Etapa 4 (i = 4).....19
j = 3.....19
j = 2.....20
j = 1.....20
j = 0.....20
Etapa 5 (i = 5).....21
j = 4.....21
j = 3.....21
Etapa 6.....21
Merge Sort.....22
Fundamento.....22
Complexidade.....22
Quick Sort.....29
Fundamento.....29
Complexidade.....29
pivô = 8.....29
INSERINDO PIVÔ.....30
pivô = 5.....30
INSERINDO PIVÔ.....31
pivô = 2.....31
INSERINDO PIVÔ.....31
pivô = 4.....31
INSERINDO PIVÔ.....32
pivô = 10.....32
INSERINDO PIVÔ.....32

Respostas das questões de 1 a 53 (assinalar)

1 – C
2 – B
3 – B
4 – B
5 – A
6 – D
7 – C
8 – C
9 – B
10 – B
11 – A
12 – A
13 – E
14 – A
15 – C
16 – B
17 – A
18 – B
19 – E
20 – C
21 – E
22 – D
23 – C
24 – B
25 – D
26 – B
27 – ERRADO
28 – A
29 – E
30 – ERRADO
31 – A
32 – A
33 – A
34 – C
35 – C
36 – C
37 – C
38 – B
39 – C
40 – E
41 – C
42 – E
43 – CERTO
44 – D
45 – D
46 – C
47 – D
48 – B
49 – C
50 – D
51 – B
52 – C
53 – B

54)

Método Bolha

Fundamento

Compara elementos adjacentes
Troca se estiverem fora de ordem
A cada passada, o maior elemento “sobe” para o final

Complexidade

Pior caso: $O(n^2)$
Caso médio: $O(n^2)$
Melhor caso: $O(n)$

Valor	8	5	3	2	1	7
Índice	0	1	2	3	4	5

tam = 6

```
for (int etapa = 1; etapa < tam; etapa++){
        troca = false;
```

Etapa = 1

```
        for (int i = 0; i < tam-etapa; i++){
```

---

i = 0

Loop do for: 0 < 6 – 1 → TRUE // i < tam - etapa

if(8 > 5) → TRUE // if(vetor[i] > vetor[i+1]){

aux = 8 // aux = vetor[i];
vetor[0] = 5 // vetor[i] = vetor[i+1];
vetor[1] = 8 // vetor[i+1] = aux;
troca = true;

Pré-visualização do vetor:

Valor	5	8	3	2	1	7
Índice	0	1	2	3	4	5

---

i = 1

Loop do for: 1 < 6 – 1 → TRUE // i < tam - etapa

if(8 > 3) → TRUE // if(vetor[i] > vetor[i+1]){

aux = 8 // aux = vetor[i];
vetor[1] = 3 // vetor[i] = vetor[i+1];
vetor[2] = 8 // vetor[i+1] = aux;
troca = true;

Pré-visualização do vetor:

Valor	5	3	8	2	1	7
Índice	0	1	2	3	4	5

---

i = 2

Loop do for: 2 < 6 – 1 → TRUE // i < tam - etapa

if(8 > 2) → TRUE // if(vetor[i] > vetor[i+1]){

aux = 8 // aux = vetor[i];
vetor[2] = 2 // vetor[i] = vetor[i+1];
vetor[3] = 8 // vetor[i+1] = aux;
troca = true;

Pré-visualização do vetor:

Valor	5	3	2	8	1	7
Índice	0	1	2	3	4	5

---

i = 3

Loop do for: 3 < 6 – 1 → TRUE // i < tam - etapa

if(8 > 1) → TRUE // if(vetor[i] > vetor[i+1]){

aux = 8 // aux = vetor[i];
vetor[3] = 1 // vetor[i] = vetor[i+1];
vetor[4] = 8 // vetor[i+1] = aux;
troca = true;

Pré-visualização do vetor:

Valor	5	3	2	1	8	7
Índice	0	1	2	3	4	5

---

i = 4

Loop do for: 4 < 6 – 1 → TRUE // i < tam - etapa

if(8 > 7) → TRUE // if(vetor[i] > vetor[i+1]){

aux = 8 // aux = vetor[i];
vetor[4] = 7 // vetor[i] = vetor[i+1];
vetor[5] = 8 // vetor[i+1] = aux;
troca = true;

Pré-visualização do vetor:

Valor	5	3	2	1	7	8
Índice	0	1	2	3	4	5

---

i = 5

Loop do for: 5 < 6 – 1 → FALSE // i < tam – etapa

Pré-visualização do vetor:

Valor	5	3	2	1	7	8
Índice	0	1	2	3	4	5

Etapa 2

for (int i = 0; i < tam-etapa; i++){

---

i = 0

Loop do for: 0 < 6 – 2 → TRUE // i < tam - etapa

if(5 > 3) → TRUE // if(vetor[i] > vetor[i+1]){

aux = 5 // aux = vetor[i];
vetor[0] = 3 // vetor[i] = vetor[i+1];
vetor[1] = 5 // vetor[i+1] = aux;
troca = true;

Pré-visualização do vetor:

Valor	3	5	2	1	7	8
Índice	0	1	2	3	4	5

---

i = 1

Loop do for: 1 < 6 – 2 → TRUE // i < tam - etapa

if(5 > 2) → TRUE // if(vetor[i] > vetor[i+1]){

aux = 5 // aux = vetor[i];
vetor[1] = 2 // vetor[i] = vetor[i+1];
vetor[2] = 5 // vetor[i+1] = aux;
troca = true;

Pré-visualização do vetor:

Valor	3	2	5	1	7	8
Índice	0	1	2	3	4	5

---

i = 2

Loop do for: 2 < 6 – 2 → TRUE // i < tam - etapa

if(5 > 1) → TRUE // if(vetor[i] > vetor[i+1]){

aux = 5 // aux = vetor[i];
vetor[2] = 1 // vetor[i] = vetor[i+1];
vetor[3] = 5 // vetor[i+1] = aux;
troca = true;

Pré-visualização do vetor:

Valor	3	2	1	5	7	8
Índice	0	1	2	3	4	5

---

i = 3

Loop do for: 3 < 6 – 2 → TRUE // i < tam - etapa

if(5 > 7) → FALSE // if(vetor[i] > vetor[i+1]){

Pré-visualização do vetor:

Valor	3	2	1	5	7	8
Índice	0	1	2	3	4	5

---

i = 4

Loop do for: 4 < 6 – 2 → FALSE // i < tam - etapa

Pré-visualização do vetor:

Valor	3	2	1	5	7	8
Índice	0	1	2	3	4	5

Etapa 3

for (int i = 0; i < tam-etapa; i++){

---

i = 0

Loop do for: 0 < 6 – 3 →TRUE // i < tam - etapa

if(3 > 2) → TRUE // if(vetor[i] > vetor[i+1]){

aux = 3 // aux = vetor[i];
vetor[0] = 2 // vetor[i] = vetor[i+1];
vetor[1] = 3 // vetor[i+1] = aux;
troca = true;

Pré-visualização do vetor:

Valor	2	3	1	5	7	8
Índice	0	1	2	3	4	5

---

i = 1

Loop do for: 1 < 6 – 3 → TRUE // i < tam - etapa

if(3 > 1) → TRUE // if(vetor[i] > vetor[i+1]){

aux = 3 // aux = vetor[i];
vetor[1] = 1 // vetor[i] = vetor[i+1];
vetor[2] = 3 // vetor[i+1] = aux;
troca = true;

Pré-visualização do vetor:

Valor	2	1	3	5	7	8
Índice	0	1	2	3	4	5

---

i = 2

Loop do for: 2 < 6 – 3 → TRUE // i < tam - etapa

if(3 > 5) → FALSE // if(vetor[i] > vetor[i+1]){

Pré-visualização do vetor:

Valor	2	1	3	5	7	8
Índice	0	1	2	3	4	5

---

i = 3

Loop do for: 3 < 6 – 3 → FALSE // i < tam - etapa

Pré-visualização do vetor:

Valor	2	1	3	5	7	8
Índice	0	1	2	3	4	5

Etapa 4

for (int i = 0; i < tam-etapa; i++){

---

i = 0

Loop do for: 0 < 6 – 4 → TRUE // i < tam - etapa

if(2 > 1) → TRUE // if(vetor[i] > vetor[i+1]){

aux = 2 // aux = vetor[i];
vetor[0] = 1 // vetor[i] = vetor[i+1];
vetor[1] = 2 // vetor[i+1] = aux;
troca = true;

Pré-visualização do vetor:

Valor	1	2	3	5	7	8

Índice	0	1	2	3	4	5

---

i = 1

Loop do for: 1 < 6 – 4 → TRUE // i < tam - etapa

if(2 > 3) → FALSE // if(vetor[i] > vetor[i+1]){

Pré-visualização do vetor:

Valor	1	2	3	5	7	8
Índice	0	1	2	3	4	5

---

i = 2

Loop do for: 2 < 6 – 4 → FALSE // i < tam - etapa

Pré-visualização do vetor:

Valor	1	2	3	5	7	8
Índice	0	1	2	3	4	5

Etapa 5

for (int i = 0; i < tam-etapa; i++){

---

i = 0

Loop do for: 0 < 6 – 5 → TRUE // i < tam - etapa

if(1 > 2) → FALSE // if(vetor[i] > vetor[i+1]){

Pré-visualização do vetor:

Valor	1	2	3	5	7	8
Índice	0	1	2	3	4	5

---

i = 1

Loop do for: 1 < 6 – 5 → FALSE // i < tam - etapa

Pré-visualização do vetor:

Valor	1	2	3	5	7	8
Índice	0	1	2	3	4	5

Etapa 6

Como **etapa < tam** é falso, ou seja **6 < 6** é falso, encerra-se o for, e consequentemente, finaliza-se o método com o vetor já ordenado.

Vetor Ordenado

Valor	1	2	3	5	7	8
Índice	0	1	2	3	4	5

55)

Método Selection

Fundamento

Procura o menor elemento do vetor
Coloca ele na posição correta
Repete para o restante do vetor

Complexidade

Pior caso: $O(n^2)$
Caso médio: $O(n^2)$
Melhor caso: $O(n^2)$

Valor	8	5	3	2	1	7
Índice	0	1	2	3	4	5

tam = 6

Etapa 1 (i = 0)

aux = vetor[0] = 8

---

j = 1

Loop: 1 < 6 → TRUE
if(5 < 8) → TRUE

aux = 5
vetor[1] = 8

Pré-visualização do vetor:

Valor	5	8	3	2	1	7
Índice	0	1	2	3	4	5

---

j = 2

if(3 < 5) → TRUE

aux = 3
vetor[2] = 5

Pré-visualização do vetor:

Valor	3	8	5	2	1	7
Índice	0	1	2	3	4	5

---

j = 3

if(2 < 3) → TRUE

aux = 2
vetor[3] = 3

Pré-visualização do vetor:

Valor	2	8	5	3	1	7
Índice	0	1	2	3	4	5

---

j = 4

if(1 < 2) → TRUE

aux = 1
vetor[4] = 2

Pré-visualização do vetor:

Valor	1	8	5	3	2	7
Índice	0	1	2	3	4	5

---

j = 5

if(7 < 1) → FALSE

Final da etapa:
vetor[0] = aux = 1

Pré-visualização do vetor:

Valor	1	8	5	3	2	7
Índice	0	1	2	3	4	5

Etapa 2 (i = 1)

aux = 8

---

j = 2

if(5 < 8) → TRUE

aux = 5
vetor[2] = 8

Pré-visualização do vetor:

Valor	1	5	8	3	2	7
Índice	0	1	2	3	4	5

---

j = 3

if(3 < 5) → TRUE

aux = 3
vetor[3] = 5

Pré-visualização do vetor:

Valor	1	3	8	5	2	7
Índice	0	1	2	3	4	5

---

j = 4

if(2 < 3) → TRUE

aux = 2
vetor[4] = 3

Pré-visualização do vetor:

Valor	1	2	8	5	3	7
Índice	0	1	2	3	4	5

---

j = 5

if(7 < 2) → FALSE

Final:
vetor[1] = 2

Pré-visualização do vetor:

Valor	1	2	8	5	3	7
Índice	0	1	2	3	4	5

Etapa 3 (i = 2)

aux = 8

---

j = 3

Loop: 3 < 6 → TRUE
if(5 < 8) → TRUE

aux = 5
vetor[3] = 8

Pré-visualização do vetor:

Valor	1	2	5	8	3	7
Índice	0	1	2	3	4	5

---

j = 4

Loop: 4 < 6 → TRUE
if(3 < 5) → TRUE

aux = 3
vetor[4] = 5

Pré-visualização do vetor:

Valor	1	2	3	8	5	7
Índice	0	1	2	3	4	5

---

j = 5

Loop: 5 < 6 → TRUE
if(7 < 3) → FALSE

Final da etapa:
vetor[2] = aux = 3

Pré-visualização do vetor:

Valor	1	2	3	8	5	7
Índice	0	1	2	3	4	5

Etapa 4 (i = 3)

aux = vetor[3] = 8

---

j = 4

Loop: 4 < 6 → TRUE
if(5 < 8) → TRUE

aux = 5
vetor[4] = 8

Pré-visualização do vetor:

Valor	1	2	3	5	8	7
Índice	0	1	2	3	4	5

---

j = 5

Loop: 5 < 6 → TRUE
if(7 < 5) → FALSE

Final da etapa:
vetor[3] = aux = 5

Pré-visualização do vetor:

Valor	1	2	3	5	8	7
Índice	0	1	2	3	4	5

Etapa 5 (i = 4)

aux = vetor[4] = 8

---

j = 5

Loop: 5 < 6 → TRUE
if(7 < 8) → TRUE

aux = 7
vetor[5] = 8

Pré-visualização do vetor:

Valor	1	2	3	5	7	8
Índice	0	1	2	3	4	5

Final da etapa:

vetor[4] = aux = 7

Pré-visualização do vetor:

Valor	1	2	3	5	7	8
Índice	0	1	2	3	4	5

Etapa 6

Como i < tam-1 é falso, ou seja, 5 < 5 é falso, encerra-se o for.

Vetor Ordenado

Valor	1	2	3	5	7	8
Índice	0	1	2	3	4	5

---

56)

Método Insertion

Fundamento

Constrói o vetor ordenado inserindo elementos um a um
Cada elemento é colocado na posição correta dentro da parte já ordenada

Complexidade

Pior caso: $O(n^2)$ (vetor invertido)
Caso médio: $O(n^2)$
Melhor caso: $O(n)$ (já ordenado)

Valor	8	5	3	2	1	7
Índice	0	1	2	3	4	5

tam = 6

Etapa 1 (i = 1)

aux = vetor[1] = 5
j = 0

---

j = 0

Loop: j >= 0 → TRUE
if(8 > 5) → TRUE

vetor[1] = 8

Pré-visualização do vetor:

Valor	8	8	3	2	1	7
Índice	0	1	2	3	4	5

j-- → j = -1

Loop: j >= 0 → FALSE

vetor[j+1] = vetor[0] = 5

Pré-visualização do vetor:

Valor	5	8	3	2	1	7
Índice	0	1	2	3	4	5

Etapa 2 (i = 2)

aux = vetor[2] = 3
j = 1

---

j = 1

if(8 > 3) → TRUE

vetor[2] = 8

Pré-visualização do vetor:

Valor	5	8	8	2	1	7
Índice	0	1	2	3	4	5

j-- → 0

---

j = 0

if(5 > 3) → TRUE

vetor[1] = 5

Pré-visualização do vetor:

Valor	5	5	8	2	1	7
Índice	0	1	2	3	4	5

j-- → -1

Loop: j >= 0 → FALSE

vetor[0] = 3

Pré-visualização do vetor:

Valor	3	5	8	2	1	7
Índice	0	1	2	3	4	5

Etapa 3 (i = 3)

aux = 2
j = 2

---

j = 2

if(8 > 2) → TRUE

vetor[2] = 8

Pré-visualização do vetor:

Valor	3	5	8	8	1	7
Índice	0	1	2	3	4	5

j-- → 1

---

j = 1

if(5 > 2) → TRUE

vetor[1] = 5

Pré-visualização do vetor:

Valor	3	5	5	8	1	7
Índice	0	1	2	3	4	5

j-- → 0

---

j = 0

if(3 > 2) → TRUE

vetor[0] = 3

Pré-visualização do vetor:

Valor	3	3	5	8	1	7
Índice	0	1	2	3	4	5

j-- → -1

Loop: j >= 0 → FALSE

vetor[0] = 2

Pré-visualização do vetor:

Valor	2	3	5	8	1	7
Índice	0	1	2	3	4	5

Etapa 4 (i = 4)

aux = 1
j = 3

---

j = 3

Loop: j >= 0 → TRUE
if(8 > 1) → TRUE

vetor[4] = 8

Pré-visualização do vetor:

Valor	2	3	5	8	8	7
Índice	0	1	2	3	4	5

j-- → 2

---

j = 2

Loop: j >= 0 → TRUE
if(5 > 1) → TRUE

vetor[3] = 5

Pré-visualização do vetor:

Valor	2	3	5	5	8	7
Índice	0	1	2	3	4	5

j-- → 1

---

j = 1

Loop: j >= 0 → TRUE
if(3 > 1) → TRUE

vetor[2] = 3

Pré-visualização do vetor:

Valor	2	3	3	5	8	7
Índice	0	1	2	3	4	5

j-- → 0

---

j = 0

Loop: j >= 0 → TRUE
if(2 > 1) → TRUE

vetor[1] = 2

Pré-visualização do vetor:

Valor	2	2	3	5	8	7
Índice	0	1	2	3	4	5

j-- → -1

Loop: j >= 0 → FALSE

vetor[j+1] = vetor[0] = 1

Pré-visualização do vetor:

Valor	1	2	3	5	8	7
Índice	0	1	2	3	4	5

Etapa 5 (i = 5)

aux = vetor[5] = 7
j = 4

j = 4

Loop: j >= 0 → TRUE
if(8 > 7) → TRUE

vetor[5] = 8

Pré-visualização do vetor:

Valor	1	2	3	5	8	8
Índice	0	1	2	3	4	5

j-- → 3

j = 3

Loop: j >= 0 → TRUE
if(5 > 7) → FALSE

vetor[j+1] = vetor[4] = 7

Pré-visualização do vetor:

Valor	1	2	3	5	7	8
Índice	0	1	2	3	4	5

Etapa 6

Loop do for: 6 < 6 → FALSE

Vetor Ordenado

Valor	1	2	3	5	7	8
Índice	0	1	2	3	4	5

57)

Merge Sort

Fundamento

Divide o vetor em duas metades
Ordena cada metade recursivamente
Depois intercala os vetores ordenados
Estratégia: dividir para conquistar
Usa memória extra, ou seja, não é in-place

Complexidade

Pior caso: O(n log n)
Caso médio: O(n log n)
Melhor caso: O(n log n)

Valor	8	5	11	2	1	7	9	3	10	6	4
Índice	0	1	2	3	4	5	6	7	8	9	10

inicio = 0
fim = 10

chamada inicial: mergeSort(V, 0, 10)

---

inicio = 0
fim = 10

verifica: 0 >= 10 → FALSE

meio = 0 + (10 - 0) / 2
meio = 5

mergeSort (ESQUERDA): mergeSort(V, 0, 5)

pré-visualização vetor parcial = | 8 | 5 | 11 | 2 | 1 | 7 |

---

* INÍCIO DO PRINCIPAL LADO ESQUERDO *

inicio = 0
fim = 5

verifica: 0 >= 5 → FALSE

meio = 0 + (5 - 0) / 2
meio = 2

mergeSort (ESQUERDA): mergeSort(V, 0, 2)

pré-visualização vetor parcial = | 8 | 5 | 11 |

---

inicio = 0
fim = 2

verifica: 0 >= 2 → FALSE

meio = 0 + (2 - 0) / 2
meio = 1

mergeSort (ESQUERDA): mergeSort(V, 0, 1)

pré-visualização vetor parcial = | 8 | 5 |

---

inicio = 0
fim = 1
verifica: 0 >= 1 → FALSE

meio = 0 + (1 - 0) / 2
meio = 0

mergeSort (ESQUERDA): mergeSort(V, 0, 0)

pré-visualização vetor parcial = | 8 |

---

inicio = 0
fim = 0

verifica: 0 >= 0 → TRUE → RETORNA

mergeSort (DIREITA):: mergeSort(V, 1, 1)

pré-visualização vetor parcial = | 5 |

---

inicio = 1
fim = 1

verifica: 1 >= 1 → TRUE → RETORNA

Executa MERGE(V, 0, 0, 1)

Subvetor esquerdo = | 8 |
Subvetor direito = | 5 |

Resultado após merge:
| 5 | 8 |

---

mergeSort (DIREITA): mergeSort(V, 2, 2)

pré-visualização vetor parcial = | 11 |

inicio = 2
fim = 2
verifica: 2 >= 2 → TRUE → RETORNA

Executa MERGE(V, 0, 1, 2)

Subvetor esquerdo = | 5 | 8 |
Subvetor direito = | 11 |

Resultado após merge:
| 5 | 8 | 11 |

---

mergeSort (DIREITA): mergeSort(V, 3, 5)

pré-visualização vetor parcial = | 2 | 1 | 7 |

---

inicio = 3
fim = 5

verifica: 3 >= 5 → FALSE

meio = 3 + (5 - 3) / 2
meio = 4

mergeSort (ESQUERDA): mergeSort(V, 3, 4)

pré-visualização vetor parcial = | 2 | 1 |

---

inicio = 3
fim = 4

verifica: 3 >= 4 → FALSE

meio = 3 + (4 - 3) / 2
meio = 3

mergeSort (ESQUERDA): mergeSort(V, 3, 3)

pré-visualização vetor parcial = | 2 |

---

inicio = 3
fim = 3

verifica: 3 >= 3 → TRUE → RETORNA

mergeSort (DIREITA): mergeSort(V, 4, 4)

pré-visualização vetor parcial = | 1 |

---

inicio = 4
fim = 4

verifica: 4 >= 4 → TRUE → RETORNA

Executa MERGE(V, 3, 3, 4)

Subvetor esquerdo = | 2 |
Subvetor direito = | 1 |

Resultado após merge:
| 1 | 2 |

---

mergeSort (DIREITA): mergeSort(V, 5, 5)

pré-visualização vetor parcial = | 7 |

---

inicio = 5
fim = 5

verifica: 5 >= 5 → TRUE → RETORNA

Executa MERGE(V, 3, 4, 5)

Subvetor esquerdo = | 1 | 2 |
Subvetor direito = | 7 |

Resultado após merge:
| 1 | 2 | 7 |

Executa MERGE(V, 0, 2, 5)

Subvetor esquerdo = | 5 | 8 | 11 |
Subvetor direito = | 1 | 2 | 7 |

Resultado após merge:
| 1 | 2 | 5 | 7 | 8 | 11 |

* FIM DO PRINCIPAL LADO ESQUERDO *

---

* INÍCIO DO PRINCIPAL LADO DIREITO *

mergeSort (DIREITA): mergeSort(V, 6, 10)

pré-visualização vetor parcial = | 9 | 3 | 10 | 6 | 4 |

---

inicio = 6
fim = 10

verifica: 6 >= 10 → FALSE

meio = 6 + (10 - 6) / 2
meio = 8

mergeSort (ESQUERDA): mergeSort(V, 6, 8)

pré-visualização vetor parcial = | 9 | 3 | 10 |

---

inicio = 6
fim = 8

verifica: 6 >= 8 → FALSE

meio = 6 + (8 - 6) / 2
meio = 7

mergeSort (ESQUERDA): mergeSort(V, 6, 7)

pré-visualização vetor parcial = | 9 | 3 |

---

inicio = 6
fim = 7

verifica: 6 >= 7 → FALSE

meio = 6 + (7 - 6) / 2
meio = 6

mergeSort (ESQUERDA): mergeSort(V, 6, 6)

pré-visualização vetor parcial = | 9 |

---

inicio = 6
fim = 6

verifica: 6 >= 6 → TRUE → RETORNA

---

mergeSort (DIREITA): mergeSort(V, 7, 7)

pré-visualização vetor parcial = | 3 |

---

inicio = 7
fim = 7

verifica: 7 >= 7 → TRUE → RETORNA

Executa MERGE(V, 6, 6, 7)

Subvetor esquerdo = | 9 |
Subvetor direito = | 3 |

Resultado após merge:
| 3 | 9 |

---

mergeSort (DIREITA): mergeSort(V, 8, 8)

pré-visualização vetor parcial = | 10 |

---

inicio = 8
fim = 8

verifica: 8 >= 8 → TRUE → RETORNA

Executa MERGE(V, 6, 7, 8)

Subvetor esquerdo = | 3 | 9 |
Subvetor direito = | 10 |

Resultado após merge:
| 3 | 9 | 10 |

---

mergeSort (DIREITA): mergeSort(V, 9, 10)

pré-visualização vetor parcial = | 6 | 4 |

---

inicio = 9
fim = 10

verifica: 9 >= 10 → FALSE

meio = 9 + (10 - 9) / 2
meio = 9

mergeSort (ESQUERDA): mergeSort(V, 9, 9)

pré-visualização vetor parcial = | 6 |

---

inicio = 9
fim = 9

verifica: 9 >= 9 → TRUE → RETORNA

---

mergeSort (DIREITA): mergeSort(V, 10, 10)

pré-visualização vetor parcial = | 4 |

---

inicio = 10
fim = 10

verifica: 10 >= 10 → TRUE → RETORNA

Executa MERGE(V, 9, 9, 10)

Subvetor esquerdo = | 6 |
Subvetor direito = | 4 |

Resultado após merge:
| 4 | 6 |

Executa MERGE(V, 6, 8, 10)

Subvetor esquerdo = | 3 | 9 | 10 |

Subvetor direito = | 5 | 6 |

Resultado após merge:
| 3 | 4 | 6 | 9 | 10 |

* FIM DO PRINCIPAL LADO DIREITO *

---

* JUNÇÃO DOS DOIS PRINCIPAIS LADOS *

Executa MERGE(V, 0, 5, 10)

Subvetor esquerdo = | 1 | 2 | 5 | 7 | 8 | 11 |
Subvetor direito = | 3 | 4 | 6 | 9 | 10 |

Resultado após merge FINAL:
| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 |

---

58)

Quick Sort

Fundamento

Escolhe um pivô
Divide o vetor em: menores que o pivô e maiores que o pivô
Aplica recursivamente nos dois lados
Também é dividir para conquistar

Complexidade

Pior caso: $O(n^2)$
Caso médio: $O(n \log n)$
Melhor caso: $O(n \log n)$

Valor	8	5	11	2	1	7	9	3	10	6	4
Índice	0	1	2	3	4	5	6	7	8	9	10

inicio = 0
fim = 10

pivô = 8

LOOP

i = 1 → 5 < 8
temporário = | 5 | - | - | - | - | - | - | - | - | - | - |
min = 1

i = 2 → 11 > 8
temporário = | 5 | - | - | - | - | - | - | - | - | - | 11 |
max = 9

i = 3 → 2 < 8
temporário = | 5 | 2 | - | - | - | - | - | - | - | - | 11 |
min = 2

i = 4 → 1 < 8
temporário = | 5 | 2 | 1 | - | - | - | - | - | - | - | 11 |
min = 3

i = 5 → 7 < 8
temporário = | 5 | 2 | 1 | 7 | - | - | - | - | - | - | 11 |
min = 4

i = 6 → 9 > 8
temporário = | 5 | 2 | 1 | 7 | - | - | - | - | - | 9 | 11 |
max = 8

i = 7 → 3 < 8
temporário = | 5 | 2 | 1 | 7 | 3 | - | - | - | - | 9 | 11 |
min = 5

i = 8 → 10 > 8
temporário = | 5 | 2 | 1 | 7 | 3 | - | - | - | 10 | 9 | 11 |
max = 7

i = 9 → 6 < 8
temporário = | 5 | 2 | 1 | 7 | 3 | 6 | - | - | 10 | 9 | 11 |
min = 6

i = 10 → 4 < 8
temporário = | 5 | 2 | 1 | 7 | 3 | 6 | 4 | - | 10 | 9 | 11 |
min = 7

INSERINDO PIVÔ

posição do pivô = min = 7
temporário = | 5 | 2 | 1 | 7 | 3 | 6 | 4 | **8** | 10 | 9 | 11 |

LADO ESQUERDO

| 5 | 2 | 1 | 7 | 3 | 6 | 4 |

pivô = 5

LOOP

i = 1 → 2 < 5
temp = | 2 | - | - | - | - | - | - |
min = 1

i = 2 → 1 < 5
temp = | 2 | 1 | - | - | - | - | - |
min = 2

i = 3 → 7 > 5
temp = | 2 | 1 | - | - | - | - | 7 |
max = 5

i = 4 → 3 < 5

temp = | 2 | 1 | 3 | - | - | - | 7 |
min = 3

i = 5 → 6 > 5
temp = | 2 | 1 | 3 | - | - | 6 | 7 |
max = 4

i = 6 → 4 < 5
temp = | 2 | 1 | 3 | 4 | - | 6 | 7 |
min = 4

INSERINDO PIVÔ

| 2 | 1 | 3 | 4 | **5** | 6 | 7 |

SUBVETOR ESQUERDO

| 2 | 1 | 3 | 4 |

pivô = 2

LOOP

i = 1 → 1 < 2
temp = | 1 | - | - | - |

i = 2 → 3 > 2
temp = | 1 | - | - | 3 |

i = 3 → 4 > 2
temp = | 1 | - | 4 | 3 |

INSERINDO PIVÔ

| 1 | **2** | 4 | 3 |

SUBVETOR

| 4 | 3 |

pivô = 4

i = 1 → 3 < 4

INSERINDO PIVÔ

| 3 | **4** |

LADO DIREITO

| 10 | 9 | 11 |

pivô = 10

LOOP

i = 1 → 9 < 10
temp = | 9 | - | - |

i = 2 → 11 > 10
temp = | 9 | - | 11 |

INSERINDO PIVÔ

| 9 | **10** | 11 |

Resultado após junção:
| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 |